Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts

Task Description – 1 (Student Academic Data Cleaning)

Task: Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

Instructions:

- Handle missing values in marks, attendance, and department.
- Remove duplicate student records based on student ID.
- Standardize text fields such as department names.
- Encode categorical variables like gender and department.

Prompt

> "Write a Python script using pandas to clean a student academic dataset. It needs to handle missing values in 'marks', 'attendance', and 'department', remove duplicate records based on 'student_id', standardize the 'department' text fields (e.g., uppercase and strip whitespace), and encode categorical variables like 'gender' and 'department' using LabelEncoder."

Code

```python
import pandas as pd
import io
from sklearn.preprocessing import LabelEncoder

# Mock Data simulating "students.csv"
csv_data = """student_id,marks,attendance,department,gender
1,85,90, Computer Science, Male
2,,80, IT , Female
3,78,, ECE , Male
1,85,90, Computer Science, Male
4,92,95, computer science , Female
5,65,70,, Male
"""
data = pd.read_csv(io.StringIO(csv_data))

# 1. Remove duplicate records based on student_id
data = data.drop_duplicates(subset=['student_id'])

# 2. Handle missing values
data['marks'] = data['marks'].fillna(data['marks'].mean())
data['attendance'] = data['attendance'].fillna(data['attendance'].median())
data['department'] = data['department'].fillna('UNKNOWN')

# 3. Standardize text fields
data['department'] = data['department'].str.strip().str.upper()
data['gender'] = data['gender'].str.strip()

# 4. Encode categorical variables
le = LabelEncoder()
data['department_encoded'] = le.fit_transform(data['department'])
data['gender_encoded'] = le.fit_transform(data['gender'])

print("Cleaned Student Data:")
print(data.head())
```

Output

```
Cleaned Student Data:
   student_id  marks  attendance        department  gender  \
0           1   85.0        90.0  COMPUTER SCIENCE    Male
1           2   80.0        80.0                IT  Female
2           3   78.0        85.0               ECE    Male
4           4   92.0        95.0  COMPUTER SCIENCE  Female
5           5   65.0        70.0           UNKNOWN    Male

   department_encoded  gender_encoded
0                   0               1
1                   2               0
2                   1               1
4                   0               0
5                   3               1
```

Explanation

The script starts by removing duplicate rows to ensure each student is represented only once. It then fills missing numerical grades with the mean/median and standardizes text by stripping whitespace and converting to uppercase, concluding by transforming categorical text data into machine-readable numeric codes.

Task Description – 2 (Financial Transaction Data Preprocessing)

Task: Preprocess financial transaction data using AI-generated Python code.

Instructions:

- Convert transaction date columns to datetime format.
- Create derived features such as transaction month and year.
- Normalize the transaction amount column.
- Identify and handle extreme transaction values.

Prompt

> "Write a Python script using pandas and sklearn to preprocess financial transaction data. I need to convert 'transaction_date' to a datetime object, extract 'transaction_month' and 'transaction_year' as new columns, handle extreme outliers in the 'amount' column using a capping method (99th percentile), and normalize the 'transaction_amount' using MinMaxScaler."

Code

```python
import pandas as pd
import numpy as np
import io
from sklearn.preprocessing import MinMaxScaler

# Mock Data simulating "transactions.csv"
csv_data = """transaction_id,transaction_date,transaction_amount
T1,2023-01-15,150.50
T2,2023-01-20,200.00
T3,2023-02-10,50.25
T4,2023-02-25,99999.99
T5,2023-03-05,300.00
"""
transactions = pd.read_csv(io.StringIO(csv_data))

# 1. Convert to datetime format
transactions['transaction_date'] = pd.to_datetime(transactions['transaction_date'])

# 2. Create derived features (month and year)
transactions['transaction_month'] = transactions['transaction_date'].dt.month
transactions['transaction_year'] = transactions['transaction_date'].dt.year

# 3. Identify and handle extreme transaction values (Capping at 95th percentile for mock data)
upper_limit = transactions['transaction_amount'].quantile(0.95)
transactions['transaction_amount'] = np.where(transactions['transaction_amount'] > upper_limit, upper_limit, transactions['transaction_amount']

# 4. Normalize the transaction amount
scaler = MinMaxScaler()
transactions['normalized_amount'] = scaler.fit_transform(transactions[['transaction_amount']])

print(transactions.info())
print("\nTransformed Data:")
print(transactions.head())
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   transaction_id      5 non-null      object
 1   transaction_date    5 non-null      datetime64[ns]
 2   transaction_amount  5 non-null      float64
 3   transaction_month   5 non-null      int32
 4   transaction_year    5 non-null      int32
 5   normalized_amount   5 non-null      float64
dtypes: datetime64[ns](1), float64(2), int32(2), object(1)
memory usage: 332.0+ bytes
None

Transformed Data:
  transaction_id transaction_date  transaction_amount  transaction_month  \
0             T1       2023-01-15             150.500                  1
1             T2       2023-01-20             200.000                  1
2             T3       2023-02-10              50.250                  2
3             T4       2023-02-25           80059.992                  2
4             T5       2023-03-05             300.000                  3

   transaction_year  normalized_amount
0              2023           0.001253
1              2023           0.001872
2              2023           0.000000
3              2023           1.000000
4              2023           0.003121
```

Explanation

We first parse the raw date strings into actual datetime objects to easily extract the month and year as new features. After capping massive outlier transactions to prevent them from skewing the data, we apply Min-Max scaling to compress all monetary values into a standard range between 0 and 1.

Task 3 – (Environmental Sensor Data Preparation)

Task: Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

- Handle missing values in temperature, humidity, and air_quality_index.
- Scale numerical features using standard scaling.
- Encode categorical fields such as sensor_status.
- Split the dataset into training and testing subsets.

Prompt

"Create a Python script using pandas and sklearn to clean environmental sensor data. Impute missing values for 'temperature', 'humidity', and 'air_quality_index' using the median. Use StandardScaler to scale these numerical features, encode the 'sensor_status' categorical column, and split the dataset into 80% training and 20% testing subsets."

Code

```python
import pandas as pd
import io
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer

# Mock Data simulating "sensor_data.csv"
csv_data = """temperature,humidity,air_quality_index,sensor_status
22.5,45.0,50,Active
23.1,,55,Maintenance
,48.0,60,Active
21.8,42.5,,Offline
24.0,50.0,70,Active
"""
data = pd.read_csv(io.StringIO(csv_data))

num_cols = ['temperature', 'humidity', 'air_quality_index']

# 1. Handle missing values
imputer = SimpleImputer(strategy='median')
data[num_cols] = imputer.fit_transform(data[num_cols])

# 2. Encode categorical fields
le = LabelEncoder()
data['sensor_status_encoded'] = le.fit_transform(data['sensor_status'])

# 3. Scale numerical features
scaler = StandardScaler()
data[num_cols] = scaler.fit_transform(data[num_cols])

# 4. Split dataset into training and testing subsets
X = data.drop(['sensor_status', 'sensor_status_encoded'], axis=1)
y = data['sensor_status_encoded']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training Data Shape: {X_train.shape}")
print(f"Testing Data Shape: {X_test.shape}")
print("\nSample Processed Features (X_train):")
print(X_train.head(2))
```

Output

```
Training Data Shape: (4, 3)
Testing Data Shape: (1, 3)

Sample Processed Features (X_train):
   temperature  humidity  air_quality_index
4     1.604931  1.407711           1.733690
2    -0.055342  0.625650           0.226134
```

Explanation

Missing sensor readings are safely populated using the median value to avoid distortion from potential sensor spikes. Numerical readings are then standardized so they share a common scale (mean of 0, variance of 1), and the data is split to properly evaluate predictive algorithms later.

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

Scenario: A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.

Requirements:

- Standardize road and location names.
- Fill missing traffic density values using appropriate statistical methods.
- Remove duplicate sensor readings.
- Normalize speed and vehicle count metrics.
- Generate a brief summary comparing dataset quality before and after cleaning.

Prompt

> "Develop a Python script using pandas to clean smart city traffic data. Standardize 'road_name' and 'location_name' columns. Fill missing 'traffic_density' values using the median. Remove duplicate sensor readings. Normalize 'speed' and 'vehicle_count' using Min-Max scaling, and print a summary comparing the dataset's describe() output before and after these cleaning steps."

Code

```python
import pandas as pd
import io
from sklearn.preprocessing import MinMaxScaler

# Mock Data simulating "traffic_data.csv"
csv_data = """sensor_id,road_name,location_name,traffic_density,speed,vehicle_count
S1, Main st , North Zone, 75, 45, 120
S2, Oak Avenue, south zone,, 30, 80
S1, Main st , North Zone, 75, 45, 120
S3, Pine Rd, East ZONE, 90, 60, 200
S4, main st, north zone, 60, 50, 150
"""
traffic = pd.read_csv(io.StringIO(csv_data))

print("--- BEFORE CLEANING SUMMARY ---")
print(traffic.describe())

# 1. Remove duplicate sensor readings
traffic_clean = traffic.drop_duplicates()

# 2. Standardize road and location names
traffic_clean['road_name'] = traffic_clean['road_name'].str.strip().str.title()
traffic_clean['location_name'] = traffic_clean['location_name'].str.strip().str.title()

# 3. Fill missing traffic density values
median_density = traffic_clean['traffic_density'].median()
traffic_clean['traffic_density'] = traffic_clean['traffic_density'].fillna(median_density)

# 4. Normalize speed and vehicle count metrics
scaler = MinMaxScaler()
traffic_clean[['speed', 'vehicle_count']] = scaler.fit_transform(traffic_clean[['speed', 'vehicle_count']])

print("\n--- AFTER CLEANING SUMMARY ---")
print(traffic_clean.describe())
print("\nCleaned Head:")
print(traffic_clean.head())
```

Output

```
--- BEFORE CLEANING SUMMARY ---
       traffic_density      speed  vehicle_count
count         4.000000   5.000000       5.000000
mean         75.000000  46.000000     134.000000
std          12.247449  10.839742      44.497191
min          60.000000  30.000000      80.000000
25%          71.250000  45.000000     120.000000
50%          75.000000  45.000000     120.000000
75%          78.750000  50.000000     150.000000
max          90.000000  60.000000     200.000000

--- AFTER CLEANING SUMMARY ---
       traffic_density     speed  vehicle_count
count         4.000000  4.000000       4.000000
mean         75.000000  0.541667       0.479167
std          12.247449  0.416667       0.421500
min          60.000000  0.000000       0.000000
25%          71.250000  0.375000       0.250000
50%          75.000000  0.583333       0.458333
75%          78.750000  0.750000       0.687500
max          90.000000  1.000000       1.000000

Cleaned Head:
  sensor_id   road_name location_name  traffic_density     speed  \
0        S1     Main St    North Zone             75.0  0.500000
1        S2  Oak Avenue    South Zone             75.0  0.000000
3        S3     Pine Rd     East Zone             90.0  1.000000
4        S4     Main St    North Zone             60.0  0.666667

   vehicle_count
0       0.333333
1       0.000000
3       1.000000
4       0.583333
```

Explanation

By standardizing string capitalization and stripping whitespaces, the script merges disparate records representing the same physical location. It handles device dropouts by filling blanks with statistical medians and normalizes metrics to provide a consistent basis for real-time traffic analysis.

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

Task: Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

- Remove duplicate posts and null text entries.
- Clean text by removing URLs, special characters, and extra spaces.
- Convert text to lowercase and perform basic tokenization.
- Remove stopwords and perform stemming or lemmatization.
- Generate basic text features such as word count and sentiment score.

Prompt

> "Write a Python script using pandas, re, and TextBlob to preprocess social media text data for sentiment analysis. Remove duplicate and null posts. Clean the text by removing URLs, special characters, and extra spaces. Convert it to lowercase. Generate new features for 'word_count' and calculate a basic 'sentiment_score' representing polarity."

Code

```python
import pandas as pd
import re
import io
from textblob import TextBlob

# Mock Data simulating "social_media_posts.csv"
csv_data = """post_id,text
1,I absolutely LOVE the new update! 🚀 Check it out: [http://link.com](http://link.com)
2,Terrible customer service today... very disappointed. #badservice
1,I absolutely LOVE the new update! 🚀 Check it out: [http://link.com](http://link.com)
3,
4,Just a normal day in the city. Nothing special.
5,    Wow!!! This    is    amazing...
"""
posts = pd.read_csv(io.StringIO(csv_data))

# 1. Remove duplicate and null posts
posts = posts.dropna(subset=['text'])
posts = posts.drop_duplicates()

# 2 & 3. Clean text function
def clean_text(text):
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Remove URLs
    text = re.sub(r'\W', ' ', text) # Remove special characters
    text = re.sub(r'\s+', ' ', text).strip() # Remove extra spaces
    text = text.lower() # Convert to lowercase
    return text

posts['cleaned_text'] = posts['text'].apply(clean_text)

# Tokenization (splitting by space for simplicity as requested by basic tokenization)
posts['tokens'] = posts['cleaned_text'].apply(lambda x: x.split())

# 5. Generate basic text features
posts['word_count'] = posts['tokens'].apply(len)
posts['sentiment_score'] = posts['cleaned_text'].apply(lambda x: TextBlob(x).sentiment.polarity)

print(posts[['text', 'cleaned_text', 'word_count', 'sentiment_score']].head())
```

Output

```
                                                text  \
0  I absolutely LOVE the new update! 🚀 Check it o...
1  Terrible customer service today... very disapp...
4    Just a normal day in the city. Nothing special.
5                 Wow!!! This    is    amazing...

                                       cleaned_text  word_count  \
0     i absolutely love the new update check it out           9
1  terrible customer service today very disappoin...           7
4     just a normal day in the city nothing special           9
5                              wow this is amazing           4

   sentiment_score
0         0.318182
1        -0.987500
4         0.253571
5         0.350000
```

Explanation

This script acts as an NLP pipeline that strips out conversational noise like emojis, URLs, and excessive punctuation to isolate the core text. It then extracts quantifiable features by counting the remaining words and leveraging TextBlob to calculate a sentiment polarity score indicating how positive or negative the post is.